# GSoC 2026 Proposal: Standardizing the Publication, Preservation, and Reusability of OmniFold Unfolding Results

## Developing a Python Framework for HDF-5 Based Data+Weight specification and HEPData Integration

**Name:** Ayachi Padmanabh Mishra
**University:** IIT Delhi Abu Dhabi (2nd Year B.Tech. CSE)
**Organisation:** HEP Software Foundation
**Location (Summer 2026):** India (UTC+5:30)
**Mentors:** Tanvi Wamorkar, Ben Nachman

---

## Table of Contents

*1. Executive Summary:* ..... *2*
*2. Project Core* ..... *3*
**2.1. Background & Motivation** ..... **3**
**2.2. My Approach:** ..... **4**
**2.3. Tasks and Proposed Deliverables** ..... **4**
*2.3.1. metadata_system (For Metadata and Naming Conventions) [Time: 1 week ]* ..... 4
*2.3.2. Elaborate Weight naming convention and observable definitions: (2 Weeks)* ..... *6*
*2.3.3. A Unified Pip installable Python API with CLI(Command Line Interface) as a stretch goal: [Time: 2 weeks]* ..... 8
*2.3.4. Model & Training Details Standardization (1 Week)* ..... 8
*2.3.5. Validation Suite(2 weeks)* ..... 9
*2.3.6. Example implementations and notebooks(1 week):* ..... 10
**2.4. Project Roadmap & Milestones** ..... **11**
*2.4.1. Community Bonding Period (May 4 – May 24) and Timeline* ..... 11
*2.4.2. Continuous Feedback & Quality Control* ..... 12
*3. About Me* ..... *13*
**Project-Specific Research & Commitment:** I have focused my preparation specifically on this problem space to ensure the proposed system is both technically sound and practically usable: ..... 13
*4. Contributions:* ..... *14*
*5. Logistics:* ..... *14*
**5.1. Time Commitment & Availability** ..... **14**
*6. Communication Strategy:* ..... *14*
*7. References:* ..... *15*
*8. Other Links* ..... *15*

# 1. Executive Summary:

Detector inaccuracies distort high-dimensional detector data from the Large Hadron Collider. Unfolding is the process of recovering underlying true particle-level distributions by correcting these inaccuracies. The standard way to visualise an observable (eg, Jet Mass) without detector effects is through histogram unfolding, where the scientist chooses a quantity to unfold and a binning rule. He then computes the histogram. While this works for that quantity, we lose all information about other observables in this process. Also, unfolding all observables together is not feasible, as it requires matrix inversion, which has abnormally high computational costs.

OmniFold solves this at the event level by using neural networks to output weights attached to each event. Because OmniFold unfolds the entire phase space, its main advantage is that binning bias gets removed entirely. Hence, once unfolded, we can conveniently multiply each event by its OmniFold weight to approximate the correct truth-level data, free of detector effects. However, these weights are tied to the experimental setup, including the specific physical process (e.g., Z+ jets), the specific detector setup being corrected for (ATLAS, CMS, etc.), and the model training details, and are therefore not universally reusable. Due to the lack of a standardized convention for describing these conditions, these weights are often unusable, as they pose a risk to any scientist unaware of all the conditions. This was also noted by Canelli et al. [1] in their paper on unbinned unfolding.

The main objective of this project is to bridge the gap between what OmniFold promises: full phase space unfolding of observables, and what scientists expect: easy publishing and preservation of results, and their efficient reuse.

Firstly, I aim to formalise the specification of these conditions into a derived YAML metadata schema coupled with the OmniFold weights dataset. This will involve defining a minimal specification, such as the process name, fiducial cuts, and detector configurations, versus a full specification that includes training details (e.g., hyperparameters, model architecture, such as the number of layers and neurons, activation functions, etc.). I will also include validation tests and the enforcement of this metadata using schemas and the schema Python library.

The second aim of this process is to release a Python API on OmniFold to apply/compute weights in a standardized format, compute observables, and plot histograms, along with references on how to integrate this data with HEPData Archives.

Lastly, I will add an example Jupyter Notebook that explains end-to-end workflows. As a 4th-semester computer science student with growing experience designing schemas and data pipelines, I aim to make OmniFold results easier to use not just for their original authors but also for the wider community that wants to build on them.

# 2. Project Core

## 2.1. Background & Motivation

Detectors distort particle-level collision data due to measurement inaccuracies. Thus, the recorded measurements are not the "true" representation of the particle-level distributions that a physicist cares about. Unfolding is the process of correcting these distortions to recover truth distributions that are comparable to theoretical predictions.

Traditional unfolding is based on the bin method, where an analyst chooses a set of observables and bin edges in advance, then corrects those histograms based on the detector response. This approach has three major limitations: Firstly, information is lost because the unfolding is tied to the fixed bins and a small number of observables. Secondly, it is not reusable because even a small change in bin size or in observable definitions may require re-doing the unfolding. Thus, the published results are often limited to a few specific histograms. Thirdly, its scope is fixed- once data is binned for an analysis, it is challenging to use the same bin for new observables.

OmniFold addresses these limitations by removing the binning process. It performs high-dimensional unfolding using machine-learning-based reweighting of simulated events. Instead of unfolding one histogram at a time, under the hood, OmniFold adjusts event weights across full phase space over multiple iterations to match detector-level simulation to data. Thus, the resulting truth-level distribution can be used to derive any observable. This makes OmniFold very powerful: a single unfolding can be reused to study many different observables and analyses without retraining.

However, this flexibility comes with its own challenges: Omnifold outputs event-level weights whose interpretation depends on detailed metadata, including event selection, preprocessing, iteration count, training details, and system encoding. Presently, there is no standard for packaging these weights, their associated datasets, and their metadata so they can be easily validated, shared, or integrated with tools like HEPData. As a result, even correct results are difficult to interpret, verify, or reuse. Therefore, much of the method's potential remains locked within individual analyses. Many other critical struggles are occurring in the scientific community, such as:

1) The popular Atlas Z+ jets study only had information about the generator in its name and had no information about the selection criteria, the model parameters, etc. This made it unusable for others who lacked context unless they were willing to review the full paper.
2) Papers such as 'tools for unbinned unfolding' by Milton et al. faced heavy documentation burdens, requiring the detailed description of preprocessing, model training, weight definitions, etc, in narrative form despite being better suited to a machine-readable metadata format.
3) Moreover, OmniFold outputs do not fit into existing HEPData records, which are designed around fixed, low-dimensional binned observables.
    a) OmniFold produces flexible, high-dimensional results where observables can be defined after unfolding. Bridging this mismatch requires defining structured datasets, metadata, and validation so that results remain both interpretable and compatible with existing infrastructure.
    b) HEPData has strict file limits (<100MB, which prevent direct upload of omnifold results.

These challenges highlight the need for standardized publication and preservation of omnifold results, forming the core motivation for this project.

## 2.2. My Approach:

I will treat Omnifold results as a well-defined data product with attributes that capture provenance (origin), correctness, and reusability.

First, I will define data and metadata specifications. On the data side, I will adopt **HDF5** as the storage format because after defining an **elaborate naming convention** that can encode weights and observables as (type)-(category)-(subcategory)-(name)-(index) [Eg, wt-statistical-bootstrap-data-3], the hierarchy of weight and observable types will be encoded in the **name itself**. This removes the need for hierarchical structures in HDF5 and allows us to represent the HDF5 file as a flat dataset safely. On the metadata side, I will add schemas to enforce these conventions, and the helper validators we design will ensure they are enforced.

Second, I will develop a pip-installable Python API with a CLI as an additional stretch goal, allowing users to easily publish, validate, and reinterpret different observables using the same OmniFold dataset. Internally, the Python API will delegate tasks to different systems, such as weight_system and validation_suite. Even without the CLI, user-facing entry points will be limited to 3 or 4 to avoid user navigation through any code complexity. The Python API I will be developing uses the schemas designed in the early weeks of the GSoC project to stress-test the data and metadata against our defined conventions. By the midpoint of the GSoC journey, I will have developed a minimal ecosystem surrounding OmniFold, ready for use to reinterpret the OmniFold dataset and plot histograms, and conveniently stress-tested by my mentors against a plethora of edge cases.

Third, I will implement a validation suite with 5 levels. The validations would include structural validation to ensure the data isn't corrupted in any way, such as the presence of NaN values. The semantic validation, on the other hand, will be used to ensure that some semantic truths surrounding OmniFold don't get violated. The most common is ensuring the weights are Non-Negative, because the final activation function can only produce **non-negative** values. Two other levels will check that the physics criteria are followed and verify the sanity of the neural network parameters, such as convergence behaviours across iterations compared with their bootstrap replicas… Finally, the most important validation level is to check the final unfolded observable after using the OmniFold weights. For this, I will mainly employ closure tests and normalisation checks.

Lastly, I will provide detailed, carefully structured Jupyter notebook demos for the entire OmniFold Ecosystem, which I am sure will be very beneficial to users. I will also conduct regular, detailed checks after the ecosystem is built to ensure the system I built is running smoothly and that HEP analysts are happy with it.

## 2.3. Tasks and Proposed Deliverables

These deliverables are the separate components that will make up the standardized OmniFold ecosystem. We first set our conventions for the overall dataset descriptions, weights, and observable definitions, along with their schemas. These will then lead to the Python API that will use these conventions and schemas to standardise the use of OmniFold weights. The validation suite will then be built to validate the reinterpretation results (whether unfolded detector data actually match the truth). Lastly, we will be adding examples and documentation for users to learn from.

We first go to the metadata system for formalising metadata conventions through machine-readable schemas

### *2.3.1. Metadata_system (For Metadata and Naming Conventions) [Time: 1 week ]*

This metadata system will describe OmniFold datasets, including the physical process name, the fiducial cut criteria, generator information (and other factors that ensure dataset provenance), and dataset identification.

**Objective**: To enforce the schema on metadata through programs that throw errors and give warnings if the metadata doesn't follow the proposed schema(standard format).

**Deliverables**: A versioned schema.json file with clearly defined Required vs Optional fields (to ensure data required for reinterpretation is always present. A validator helper (validate_metadata.py) to check whether required fields are present, the schema is being enforced, and to check type consistency.

I will provide metadata conventions in a **table format before coding the schema, for checking.** The schema.py file will enforce the schema. For the YAML metadata design I had made during the evaluation task, please refer to this GitHub repo.

Table 1: Format for presenting Proposed Metadata Conventions before hardcoding them in the schema

| **Category** | **Key** | **Type** | **Requirement** | **Description** |
|---|---|---|---|---|
| **Provenance** | generator | string | **Required** | MC Generator (e.g., Pythia_8.3) |
| **Weights** | w_nominal | float64 | **Required** | Central OmniFold weight for the event |
| **Physics** | luminosity_fb | float64 | Optional | Target data integrated luminosity in **$fb^{-1}$** |

Below is a JSON schema prototype of schema.json (the 'properties' that are not marked 'required' are optional). This is a minimal schema that will be expanded on in the future. The schema is a blueprint specifying which dataset attributes every metadata record should have and which are optional.

```
{"$schema": "http://json-schema.org/draft-07/schema#",
 "title": "OmniFold Metadata Schema v1.0",
 "type": "object",
 "required": ["dataset_identification", "weights", "schema_version"],
 "properties": {
  "schema_version": { "type": "string" },
  "dataset_identification": {
   "type": "object",
   "required": ["process_name", "center_of_mass_energy"],
   "properties": {
    "process_name": { "type": "string" },
    "center_of_mass_energy": { "type": "number" }
   }},
  "weights": {
   "type": "object",
   "required": ["w_nominal"],
   "properties": {
    "w_nominal": { "type": "number" },
    "w_systematic": { "type": "array", "items": { "type": "number" } }}
```

How do we check whether a given metadata file follows our schema?

Below is validate_metadata.py. It is a minimal prototype to show the feasibility of the enforcement. This function validates the schema's enforcement. The detailed validation suite will be presented later in this proposal, including physics, semantics, and other validation levels. The main work to reach this milestone is to define the schema fully.

```
import yaml, json
from jsonschema import validate
def verify_metadata(config_path, schema_path):
    # Load human-readable YAML metadata file and the formal JSON schema
    with open(config_path) as f: config = yaml.safe_load(f)
    with open(schema_path) as f: schema = json.load(f)
    validate(instance=config, schema=schema) #throws an error if metadata does not match the schema,
    return True
```

How does the function work?

We can think of the JSON schema we defined as a blueprint, and the YAML metadata file as the building. Our core objective is to ensure that all rooms, pipes, etc. (process name, event selection criteria, uncertainties, etc.) match exactly as intended in the blueprint. Hence, we go room by room and check each thing one by one. This is exactly what JSON Schema is. The validate function does for us. This way, if something doesn't match (a required metadata field isn't present), we can raise a warning and notify the user about this uncertainty in the information.

A lot of core standardization is still left in the naming, definitions, and inclusions of weights, observables, model training details, etc. We discuss these in the upcoming sections.

---

### *2.3.2. Elaborate Weight naming convention and observable definition: (2 Weeks)*

**Why is it important**: Having standardized the metadata of our files to ensure data provenance, we still have to navigate the huge dataset simply because of how unorganized the weights, observables, and their names are. Since we have multiple types of weights (nominal, systematic, bootstrap, theory, and others), we need to standardize how they are stored and named to ensure easy programmable access. For this, we need to ensure consistency in naming and definition of these quantities.

**Description**: I will present a naming convention for both omnifold weights and observables. We will standardize the container format of weights to HDF5 and also provide an observable description format.

**Deliverables**: Description of proposed Weight Naming format with documentation on design rationale, examples, and lastly, Indexing definitions. We now divide the type of deliverables into the weight system and the observables system and discuss them in detail as follows:

1. ***Weight System Deliverables(Kept Minimal):***

**Proposed naming convention for weights and observables**:
(type)_(category)_(subcategory)_(name)_(index) [Eg:][Note: These are actually underscores, not hyphens! I am using snakecase]

**Python Utilities** (functions) to load and filter(using regex) weights.

**Rules for indexing** through weight variations(Eg, ensemble1,2,3…) and event indices, and A small walkthrough on how to filter for the required weight using *regex* (wt_demo.py)
Lastly, we provide **Documentation** on the interpretation of weight names.

2. ***Observable Definition System (Kept Minimal here too):***

Here, I will provide a schema for observable definitions that includes important fields such as name, description, formula, units, phase space, and, optionally, binning. There will again be a validator function solely to verify the observable definitions file against the defined observables schema. An example prototype is given below:

```
# This is an example of the YAML metadata I have made defining observables in OmniFold datasets (could alternatively be defined on JSON too)(it is machine-readable)
type: object
required:
  - observables

properties:
  observables:
    type: array
    items:
      type: object
      required:
        - name
        - description
        - definition
        - units
      properties:
        name:
          type: string
          pattern: "^obs__.+"
#and so on, having other fields
```

---

Having standardized most of the dataset within the first 3 weeks, we will turn our focus to getting the minimum product ready by the end of week 5. This involves having a Python API ready, which will use validations via our schemas and additional programs to mask the core routines that occur during reinterpretation.

### 2.3.3. *A Pip installable Python API with CLI(Command Line Interface) as a stretch goal: [Time: 2 weeks]*

**Description**: Users will be able to publish weights, reinterpret their datasets and plot histograms. This could also be done right from the CLI itself, if it gets completed. (It is just a stretch goal. The API will have no more than 3-4 user-facing entry points to keep it simple for users. By the end of this week, we will have an API package that lets you easily run reinterpretation on the dataset with a few API commands.

```
import OmniFold as of

# 1. Initialize from a record (e.g., a HEPData entry) or give the path to the file if held locally
analysis = of.Analysis("ins1234567")

# 2. Load the heavy weights(greatly internal optimisations
analysis.load()

# 3. Apply to a local 'events' array (NumPy or Awkward)
dist, weights = analysis.apply(my_events, "jet_pt")

# 4. Plot with automatic uncertainty propagation
.plot_unfolded(dist, weights, label="OmniFold Unfolded")
```

Being able to do the above is, in fact, the core motivation for these weeks. The deliverable will be the python api and its subprograms.

---

### 2.3.4. *Model & Training Details Standardization (1 Week)*

For both reinterpretation and full reproduction of results, a certain number of model parameters need to be known so that we can accurately describe the Neural network model that our algorithm learnt from. For reinterpretation, we only need some information about the neural network to determine whether it is appropriate. However, for full retraining of the neural network, additional information will be required.

**Deliverables:** A formal model_schema.json with a helper function for validating the dataset.

To ensure OmniFold results are both reusable and reproducible, I propose two publication tiers:

(i) **Minimal (Reinterpretation):** final weights, feature definitions, preprocessing details → allows direct usage without retraining

(ii) **Full (Reproducibility):** architecture definitions, training hyperparameters, iteration strategy, and model checkpoints → allows full retraining and pipeline reconstruction

The metadata schema will standardize the description and also what is required and Not Required for the description of the model and its configuration:

(i) **Model Architecture:** layer structure, activations, input/output dimensions

(ii) **Training hyperparameters:** batch size, optimizer, learning rate, early stopping

(iii) **Unfolding configuration:** number of iterations, step-1/step-2 setup, weight update logic

(iv) **Preprocessing (strict):** feature ordering, transformations (e.g., angular encodings), normalization parameters

Additional components that I will include for full reproducibility: random seeds (to reduce stochastic variation), ensembling configuration (number of models, aggregation method), dataset references and selection definitions, etc.

This defines a practical reproducibility standard that allows the full pipeline to be reconstructed with high accuracy while still supporting lightweight reinterpretation use cases.

---

### *2.3.5. Validation Suite(2 weeks)*

In this suite, we have to ensure that any dataset produced is physically sound, meaningful, and can be trusted by the community. This involves checking against our metadata schemas to ensure the dataset is semantically and physically sound. And then also doing checks on the practical side, like checking the convergence trend of the training.

My strategy: I propose validating at 5 levels, each with clearly defined checks and explicit pass/fail criteria (defining these criteria will be a core part of this deliverable). The validation will be fail-first, whereas the dataset is checked meaningfully only if it is structurally sound and has no errors in its metadata or contents.

Deliverables: **Validation.py**, which includes a class with all 5 validations below as methods, and a GOLDEN dataset that always passes the validation suite. (This will be helpful while testing the validators)

We have the 5 levels of validations with the tasks as follows:

(i) **Structural Validation:** Ensure metadata schema is enforced,

(ii) **Semantic Validation:** Ensure observable scales and ranges are consistent

(iii) **Physics-Level Validation:** Verify that all events satisfy fiducial cuts defined in metadata and ensure no observables violate selection criteria while still being present in the dataset

(iv) **Neural Network-Level Validation:** Check that OmniFold output weights are physically valid (e.g., non-negative given final activation) and Stability across iterations (detect oscillations or divergence)

(v) **Final Output-Level Validation:** This is the most important level, which is only run if the dataset passed all the checks from the above levels. The rigorous tests in this are as follows:

a) Closure tests (reco vs unfolded comparisons)
b) Normalization checks (preservation of total cross-section)
c) Consistency of unfolded distributions across iterations and ensembles

Below is a prototype validation suite. Here, we used only 1 method, but in the actual suite, we will use separate methods.

```python
import numpy as np

class OmniFoldValidator:
    def __init__(self, data, metadata):
        self.data = data  # HDF5 or Array-like
        self.meta = metadata
        self.report = {}

    def validate(self):
        # 1. Structural: Check for NaNs/Infs
        weights = self.data.get('weights', np.array([]))
        self.report['structural'] = not np.isnan(weights).any() and len(weights) > 0

        # 2. Physics: Check if pT satisfies the cut defined in metadata
        pt_cut = self.meta.get('pt_min', 0)
        self.report['physics'] = np.all(self.data.get('pt', 0) >= pt_cut)

        # 3. NN Health: Ensure no extreme non-physical weight spikes
        .report['nn_health'] = np.max(np.abs(weights)) < 1e5

        return all(self.report.values()), self.report

# Usage
# success, status = OmniFoldValidator(data, meta).validate()
```

---

### 2.3.6. *Example implementations and jupyter notebooks(1 week):*

Even with all the above deliverables covered, the project will still be incomplete if we don't actually show our users how to use it. Hence, we keep a folder just for example implementations and notebooks. These are files that will include user-centric programs to show how to make the most of the package.

Deliverables:

(i) Jupyter notebooks showing the full workflows unfolding → publication → validation → reinterpretation.
(ii) We will also be providing individual scripts that will handle one task only (publish, validate, reproduce, reinterpret, upload), so users can follow the pipeline at each step too.

### 2.3.7. HEP DATA Integration Layer [Stretch Goal](2 Weeks):

This could be a very valuable addition to the project. I have designed a minimal tier of this project, which is very easy to implement for now, since integrating omnifold results directly into HEPData records is so difficult. It involves the following steps:

- From the hepdata_lib python module, create a table object and use its add_additional_resource method to link the Zenodo URL of the dataset to this. If the dataset is available at the URL, it is automatically linked to HEPData records.

· While our dataset is not technically integrated with the HEPData, it still has its own HEPData record ID.

This creates an HEPData ID for the dataset, allowing it to be searched in HEP Data's records. However, this is not the full integration that we want. Hepdata is inherently binned, whereas omnifold results are unbinned. Figuring out a way to do the core integration (if possible) will take more time and require more engineering effort.

## 2.4. Project Roadmap & Milestones

After deciding on the proposed deliverables, I have structured this timeline to complete the minimum but most important tasks within the first 3-4 weeks. By the end of the 5th week, we will have a minimal project framework, including the minimal API and all the schemas.

### *2.4.1. Community Bonding Period (May 4 – May 24) and Timeline*

During the community bonding period, I will complete the necessary groundwork to hit the ground running on Day 1. This involves the following steps to get fully primed for the development phase. It involves the following tasks to be completed:

- **Codebase Audit:** Study hep-lbdl/OmniFold and ViniciusMikuni/OmniFold codebases alongside ATLAS Z+jets GitLab implementations to finalize publication boundaries. Also, study current codebases
- **Infrastructure:** Set up the package skeleton (PEP 621), CI/CD development environment, and collect representative OmniFold outputs for testing.
- **Mentor Alignment:** Align with mentors on schema field priorities and the user-facing API design and expectations.

*12-Week Development Phase*

| **Week** | **Dates** | **Deliverable/Milestone** | **Key Actions to Achieve** |
|---|---|---|---|
| **1** | May 25 – May 31 | **Schema v1.0 & Metadata validators (DefinedSchema will now be enforced on data)** | 1. Implement YAML loader logic.<br>2. Define machine-readable constraints in schema.json<br><br>3. Implement schema validators |
| **2-3** | June 1 – June 14 | **The Weight and observable Managers** | 1. Build the weights__<cat>__…. string parser.<br>2. Enable programmatic weight discovery.<br><br>Artifacts:<br><br>1. weights schema and observable definitions schema<br><br>2. observable_validator.py and weights_validator.py, weights_loader.py |
| **4-5** | June 15 – June 28 | **Minimal Working API** | 1. Minimal working API for reinterpretation.<br><br>2. Combine validators and schemas from previous phases for<br><br>Artifacts: minimal api, minimal_validator.py |

| Week | Dates | Deliverable/Milestone | Key Actions to Achieve |
|---|---|---|---|
| **6** | June 28 – July 5<br><br>July6: midterm<br><br>Evaluation | **Tiered Model Specification and Rigorous API evaluation before midterm evaluations** | 1. Define Minimal (Reinterpretation) vs. Full (Reproducibility) requirements.<br>2. Standardize architecture/hyperparameter storage.<br>3. Implement preprocessing state serialization.<br><br>Artifacts: Schema for Training details, validate_model.py |
| **7** | July 6 – July 12 | **Structural & Semantic Validation** | 1. Build NaN/empty-data detectors.<br>2. Implement unit and scale consistency checks.<br>3. Create pass/fail logic for metadata completeness.<br><br>Artifacts: Validator methods for structural and semantic validations (use earlier |
| **8** | July 13 – July 19 | **Physics. NN and Final Output Validation** | 1. Implement fiducial cut enforcement.<br>2. Build weight sanity (non-negativity) checks.<br>3. Develop iteration stability/convergence, trackers.<br><br>(Artifacts: Validation Python class with the class having all 5 validator methods and a golden dataset for testing validators) |
| **9,10,11** | July 20 – Aug 10 | **Documentation & HEPData Integration** | 1. Bridge schema to HEPData-ready YAML (Stretch Goal).<br>2. Finalize end-to-end "Analysis-to-Archive" scripts.<br>Artifacts: End-to-end workflow example notebooks, minimal Hepdata integration, and lastly |
| **12** | Aug 11 - 18 | **Final Audit & Peer Review** | 1. Conduct rigorous peer review with mentors/colleagues.<br>2. Finalize bug fixes and stabilization. |

### *2.4.2. Continuous Feedback & Quality Control*

I will maintain a **Progress & Delta** reporting cycle to ensure full alignment with HSF standards. Every **Thursday morning**, I will submit a mid-week status report detailing current roadblocks and logic changes. After incorporating mentor feedback, I will finalize the week's deliverables by **Sunday morning (India Time)**. This schedule allows mentors to inspect the project's quality on Saturday evening (US), ensuring the following week begins with a peer-reviewed foundation.

## 3. About Me

**Ayachi Padmanabh Mishra**
B.Tech. Computer Science & Engineering, Indian Institute of Technology Delhi – Abu Dhabi
**CGPA:** 9.01/10

- **GitHub:** AyachiMishra | **LinkedIn:** Ayachi's LinkedIn
- **Patent No- 202541093222 (Indian and PCT): Ayachi P. Mishra** "*Agricultural Recommendation System and Method using hand-held soiled monitoring with geo-tagged data.*"

**Physics-Aware Computational Foundation:**
I have knowledge in both Energy Physics and data systems, combining my strong CS background with solid training in physics and mathematics (A grades in Quantum Physics, Electrodynamics, Linear Algebra, Probability, and ML)
This allows me to understand omnifold beyond a black-box workflow, as reflected in my detailed explanations and notes developed during the evaluation tasks (task files).

**Proven Architecture & Documentation Standards:** I bring a structured, documentation-first approach to system design:

> **Protocol & Schema Design:** In a specialized hackathon for Mashreq Bank, I developed a 4-module Python pipeline and designed a **JSON-based governance schema** to ensure data consistency and safe handling across workflows (it primarily used a waterfall model where the JSON schema will grow module after module until the final 4th module)Project Repository.
>
> **Formal Intellectual Property:** I have filed a patent on a physics-based device that predicts and recommends fertilization, irrigation, and other requirements for a crop. It requires precise documentation and iterative coordination with the patent attorney. This experience directly informs my approach to defining robust and reusable publication standards.

**Experience With Neural Networks:** In my project on predicting building heating loads, I redefined new building features, such as compression ratio and glazing area distribution, to enable smoother convergence during training. This directly helped me understand how to test for convergence and the procedures for formally evaluating neural networks. The dataset was from the University of California, Irvine.

In my invention, I have also experimented with neural network-based approaches in my precision agriculture system to model relationships between sensor inputs and soil parameters. This experience helped me understand practical aspects of training models on structured, time-dependent data.

**Domain Immersion:** Studied OmniFold through Dr. Ben Nachman's lectures and reviewed key research papers(mainly the abstract and conclusion, and if possible, the summary of the methodology used).

- OmniFold Lecture(SMU Physics department speaker series)
- Paper (Practical Guide to Unbinned Unfolding)

**My motivation:** I have always been intrigued by the idea of new particles and undiscovered physics. The idea of stress testing the standard model is very fascinating to me. Being part of the standardization of this OmniFold procedure, which would enable 100s of scientists to use the algorithm, feels like a lifetime opportunity. It is like having a real impact in the scientific world, which would be remembered through the widespread use of this algorithm and its defined standards.

# 4. Contributions:

**ROOT (RooFit) – Numerical Stability Fix**
Identified a division-by-zero issue in RooPlot::chiSquare when bins have zero error (e.g., expected data errors), leading to NaN values in pull calculations.

1) Implemented a safeguard to handle zero-error bins and added a user-facing warning.
2) Contributed as an interim fix while the method transitions toward RooAbsReal::createChi2

PR: *Guard against zero bin errors in chiSquare calculation* (#21710) (Open)

**Model Reusse in Omnifold Iterations – Clarification On Intended Behaviour**
Investigated the training behavior in the official repo for OmniFold examples (GaussianExample.ipynb).

1) Observed that the same neural network instance is reused across steps and iterations without weight reinitialization.
2) Raised a technical query on whether this reuse is intentional (e.g., for efficiency) or deviates from the expected independence in iterative density ratio estimation.
3) Highlighted potential implications for convergence and correctness.

Issue: *Model reuse across iterations in OmniFold* (#18) (Open)

# 5. Logistics:

## 5.1. Time Commitment & Availability

I am committing to a **45-hour-50-hour/week professional immersion** for the duration of the 11-week GSoC period. While the core development is estimated at 175 hours, I have allocated a significant "Research & Validation Buffer" to account for the deep literature review required for HEPData schema mapping and rigorous edge-case testing.

My schedule is designed for high-intensity deep work blocks, typically 6–7 hours on weekdays and 4–5 hours on weekends. This flexibility ensures I can investigate deeply before starting work on the proposed deliverable. This is my exclusive professional commitment for the summer; I have no conflicting internships or travel plans.

# 6. Communication Strategy:

I am actually comfortable with any mode of communication that the mentors prefer. However, I will suggest avoiding email threads as a mode of communication, as emails can easily get lost in our inboxes.Some suggestions:

1. For face-to-face communication: Google Meet, Zoom, etc.
2. For messaging communication: Mattermost, Slack, or anything other than an email thread.
3. Code-centric discussions: These can be done through issues/pull requests at the GitHub repo.

The "Thursday Delta": I will provide a weekly summary of progress ($\Delta$), current issues, and next steps every Thursday to ensure the project remains on track before the weekend.

## 7. References:

[1] Canelli, F., Cormier, K., Cudd, A., Gillberg, D., Huang, R. G., Jin, W., ... & Acosta, F. T. (2026). A practical guide to unbinned unfolding. *The European Physical Journal C*, *86*(2), 106.

[2] Andreassen, A., Komiske, P. T., Metodiev, E. M., Nachman, B., & Thaler, J. (2020). OmniFold: a method to simultaneously unfold all observables. *Physical Review Letters*, *124*(18), 182001.

## 8. Other Links

**1.** Documentation used to understand the usage of the hepdata_lib library to read data into Python list/array format and write data from Python lists to YAML- based format: Hepdata_lib.docs

**2.** GitHub**:** AyachiMishra

**3.** LinkedIn**:** Ayachi's LinkedIn

4. Ayachi's Repository for Omnifold: Project Repository

5. Mr. Nachman's lecture on Omnifold: OmniFold Lecture(SMU Physics department speaker series)

6. Ayachi P. Mishra, Indian Patent No 202541093222 and PCT patent no. **PCT/IB2025/061635,** **"***Agricultural Recommendation System and Method using hand-held soiled monitoring with geo-tagged data.":* Patent Portal